Guía de Entrenamiento y Reentrenamiento

Microservicio Auditor Recepción

Módulo: Auditor Recepción **Tipo:** Guía Operativa ML **Estado:** Oficial

1. Requisitos Previos y Entorno

Asegúrese de contar con las siguientes dependencias e infraestructura antes de iniciar el proceso:

- **Python:** 3.10 o superior
- **Librerías Clave:** `scikit-learn`, `pandas`, `numpy`, `xgboost`, `onnxruntime`, `joblib`, `pydantic`.
- **Acceso a Datos:** Lectura de la base de datos de históricos de recepción (`casalaiv.7.2.sql` / respaldo de auditorías).
- **Espacio en Disco:** Mínimo 2 GB libres para datasets temporales y artefactos del modelo.

2. Preparación y Limpieza de Datos

El script de preprocesamiento extrae las métricas históricas de recepción, incluyendo pesos, dimensiones, estados de empaque, diferencias contra orden de compra y tiempos de tránsito.

```
# Ejecutar pipeline de extracción y limpieza de características
python -m src.pipelines.extract_auditoria_data \
    --input-db backup/casalaiv.7.2.sql \
    --output-data data/processed/auditor_recepcion_dataset.csv
```

Reglas de Limpieza Aplicadas:

1. Imputación de valores nulos mediante la mediana por categoría de producto.
2. Normalización de variables continuas (`StandardScaler`).
3. Codificación One-Hot para variables categóricas (`proveedor_id`, `tipo_empaque`, `zona_recepcion`).
4. Eliminación de outliers extremos mediante algoritmo Isolation Forest.

3. Ejecución del Entrenamiento

El entrenamiento se realiza ejecutando el módulo principal de entrenamiento con validación cruzada estratificada (Stratified K-Fold, $_{k=5}$).

```
python -m src.models.train_auditor \
    --dataset data/processed/auditor_recepcion_dataset.csv \
    --model-type xgboost \
    --config configs/auditor_recepcion_config.yaml \
    --output-dir models/artifacts/
```

Estructura de Salida Generada (`models/artifacts/`):

- `model_auditor_recepcion.pkl`: Artefacto serializado base.
- `model_auditor_recepcion.onnx`: Versión optimizada para baja latencia en inferencia.
- `scaler_auditor.pkl`: Escalador para el preprocesamiento en la API.
- `metrics_report.json`: Reporte cuantitativo de resultados.

4. Validación de Métricas y Umbrales Mínimos

Antes de autorizar el despliegue a producción, ejecute la suite de evaluación automatizada para verificar que el modelo cumple con las tolerancias requeridas:

```
python -m src.models.evaluate \
    --model-path models/artifacts/model_auditor_recepcion.onnx \
    --test-data data/processed/test_set.csv
```

Criterios de Aceptación (Quality Gate):

Métrica / Parámetro	Umbral Requerido	Acción en Caso de Fallo
Precision (Global)	≥ 95.0%	Revisar ponderación de clases y ajustar hiperparámetros.
F1-Score (Clase Rechazado)	≥ 93.0%	Incrementar muestras de entrenamiento para la clase Rechazado.
Latencia (p95)	≤ 50 ms	Optimizar grafo en ONNX u omitir variables redundantes.
Tamaño del Artefacto	≤ 150 MB	Aplicar cuantización (FP16 / INT8) sobre el modelo.

5. Empaquetado y Despliegue en Producción

Una vez validadas las métricas en el Quality Gate, proceda con la actualización de los artefactos en el microservicio:

1. **Copiar Artefactos al Microservicio:**

   ```
   cp models/artifacts/model_auditor_recepcion.onnx services/auditor_recepcion/app/ml/
   cp models/artifacts/scaler_auditor.pkl services/auditor_recepcion/app/ml/
   ```

2. **Ejecutar Pruebas Unitarias e Integración:**

   ```
   pytest services/auditor_recepcion/tests/
   ```

3. **Reiniciar o Desplegar la Imagen Docker:**

   ```
   docker build -t microservicio-auditor-recepcion:v2.4.0 services/auditor_recepcion/
   docker-compose up -d --build auditor-recepcion
   ```

6. Monitoreo Post-Despliegue y Concept Drift

- **Verificación de Logs:** Monitorear respuestas HTTP 5xx e indicadores de fallback en Prometheus / Grafana.
- **Drift de Datos:** Se activa una alerta automática cuando la distribución de diferencias en peso difiera en más de 8% en relación al dataset de baseline durante un periodo de 7 días continuos.